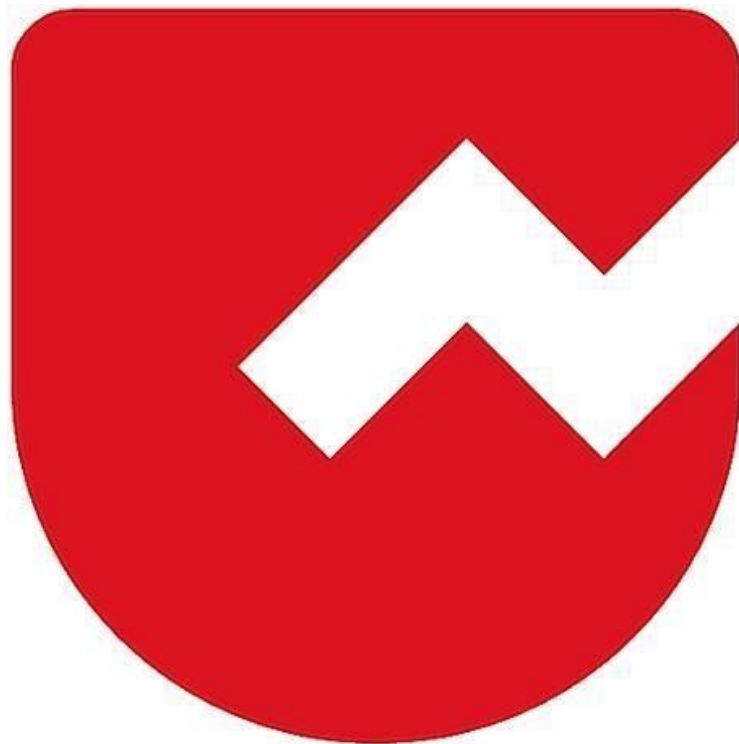




Zoho Schools for Graduate Studies



Notes

Java I/O Sources

Session Date: December 4, 2025

1. Introduction to Java I/O

Java I/O (Input/Output) is a system that allows programs to read data from Sources and write data to destinations. These sources can include files, keyboard input, network connections, memory, etc.

2. Streams in Java

Streams represent a flow of data. They are of two main types:

A. Byte Streams (8-bit data)

- Classes: FileInputStream, FileOutputStream
- Used for binary data such as images, audio files, PDFs, etc.

B. Character Streams (16-bit Unicode)

- Classes: FileReader, FileWriter
- Used for reading and writing text files.

3. Types of I/O Sources

Java can perform I/O operations on multiple sources:

A. Console / Keyboard

- Using System.in, System.out, Scanner
- Reads user input from the terminal.

B. Files

- FileInputStream, FileOutputStream, BufferedReader, FileWriter
- Most commonly used for persistent data storage.

C. Network

- Using Sockets (TCP/UDP)
- `socket.getInputStream()`, `socket.getOutputStream()`

D. Memory Streams

- `ByteArrayInputStream`
- `CharArrayReader`
- Used when data is already present in memory.

The Solution

```
public class ByteArrayDemo {  
    public static void main(String[] args) {  
        byte[] b="RajaGanapathy likes Briyani".getBytes();  
        ByteArrayInputStream bis=new ByteArrayInputStream(b);  
        String destPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/  
        FileOutputStream fos=new FileOutputStream(destPath);  
        int value=bis.read();  
        while(value!=-1) {  
            fos.write(value);  
            value=bis.read();  
        }  
        bis.close();  
        fos.close();  
        System.out.println("Byte Array contents are copied successfully!");  
    }  
}
```

4. Buffered Streams (Decorator Pattern)

Buffered streams improve performance by reducing the number of I/O operations.

Examples:

- BufferedInputStream
- BufferedOutputStream
- BufferedReader
- BufferedWriter

They wrap around basic streams and make reading and writing faster.

5. Difference Between Stream Types

- InputStream / OutputStream → handles raw bytes
- Reader / Writer → handles characters (text)

Examples:

- To read a text file: use FileReader
- To read a binary file: use FileInputStream

```
import java.io.*;

// Develop a Java program to copy the contents of a file

public class FileCopyDemo {
    public static void main(String[] args) throws Exception {
        String srcPath="/Users/selvam-8490/Desktop/EclipseProjects/GS20252/src/p
        FileInputStream fis=new FileInputStream(srcPath);
        FileOutputStream fos=new FileOutputStream("t2.txt");
        int value=fis.read();
        while(value!=-1) {
            fos.write(value);
            value=fis.read();
        }
        fis.close();
        fos.close();
        System.out.println("File contents are copied successfully!");
    }
}
```

Input source – Generates Stream

- File (FileInputStream)
- Keyboard (System.in)
- Byte Array (ByteArrayInputStream)
- Object (ObjectInputStream)

Output Source – Consumes Stream

- Monitor (System.out)
- File (FileOutputStream)
- ObjectOutputStream

Stream : A sequential flow of data

Classification of Stream:

- Byte Oriented Stream → read/write byte by byte
- Character Oriented Stream → read/write character by character

6. Important Points

- Streams are the core mechanism for data flow in Java
- Java supports both byte and character operations
- Multiple I/O sources: console, file, network, memory
- Buffers significantly speed up I/O processing